

RideWithPals Chat Socket.io API Documentation

Comprehensive Socket.io documentation for Frontend and Mobile App developers integrating Chat (1-on-1 Direct Messaging & Activity Group Chat).

1. Socket Connection & Authentication

Connection URL:

```
http://<YOUR_SERVER_HOST>:<PORT>
```

Authentication:

Pass the JWT Token in `handshake.headers` or `handshake.query` :

```
// Example Client Connection (Socket.io Client JS / React Native / Flutter)
const socket = io("https://api.ridewithpals.com", {
  headers: {
    Authorization: `Bearer ${userJwtToken}`
  },
  // OR via query params:
  // query: { token: userJwtToken }
});
```

2. Data Models & Structures

A. User Object (`User`)

```
interface User {
  id: number;
  fullName: string;
  profileImage: string | null;
}
```

B. Chat Thread Model (`ChatThread`)

```
interface ChatThread {
  id: number;
  type: "direct" | "activity"; // "direct" for 1-on-1 DM, "activity" for Group Chat
  userOneId?: number | null;
  userTwoId?: number | null;
  rideId?: number | null;
  lastMessageAt: string; // ISO Date String
  createdAt: string;
  updatedAt: string;
  isGroup?: boolean;
  title?: string;
  otherUser?: User | null; // For 1-on-1 direct chats
  ride?: {
    id: number;
    rideName: string;
    clubId: number | null;
    userId: number;
  } | null; // For activity group chats
  lastMessage?: ChatMessage | null;
  unreadCount?: number;
}
```

C. Chat Message Model (ChatMessage)

```
interface ChatMessage {
  id: number;
  threadId: number;
  senderId: number;
  message: string;
  isRead: boolean;
  readAt?: string | null;
  createdAt: string;
  updatedAt: string;
  sender: User;
}
```



3. Server Events (Client Emits Server Responds via Ack)

All socket request events use standard Socket.io Acknowledgement Callbacks: `ack({ ok: boolean, data?: any, message?: string })`.

1 Get or Create Chat Thread

Opens/creates a 1-on-1 DM thread OR an Activity Group Chat thread and auto-joins the socket room.

- **Event Name:** `chat:thread:getOrCreate`
- **Emit Payload (Option 1: For Activity Group Chat):**

```
{
  "rideId": 45
}
```

- **Emit Payload (Option 2: For 1-on-1 Direct Chat):**

```
{
  "targetUserId": 12
}
```

- **Server Response (`ack` callback):**

```
{
  "ok": true,
  "data": {
    "id": 102,
    "type": "activity",
    "rideId": 45,
    "userOneId": null,
    "userTwoId": null,
    "lastMessageAt": "2026-08-06T16:45:00.000Z",
    "createdAt": "2026-08-06T16:45:00.000Z",
    "ride": {
      "id": 45,
      "rideName": "Weekend Cycling Tour",
      "clubId": 5,
      "userId": 2
    }
  }
}
```

2 Join Thread Room

Manually joins the socket room for real-time updates of a thread.

- **Event Name:** `chat:thread:join`
- **Emit Payload:**

```
{
  "threadId": 102
}
```

- **Server Response** (`ack` callback):

```
{
  "ok": true
}
```

3 Leave Thread Room

Leaves the socket room when navigating away from the chat screen.

- **Event Name:** `chat:thread:leave`
- **Emit Payload:**

```
{
  "threadId": 102
}
```

- **Server Response** (`ack` callback):

```
{
  "ok": true
}
```

4 Send Chat Message

Sends a text message to a Direct Chat or Activity Group Chat.

- **Event Name:** `chat:message:send`
- **Emit Payload (Option A - by threadId):**

```
{
  "threadId": 102,
  "message": "Hey everyone, see you at 8:00 AM at the starting point!"
}
```

- **Emit Payload (Option B - by rideId):**

```
{
  "rideId": 45,
  "message": "Hey everyone, see you at 8:00 AM at the starting point!"
}
```

```
}
```

- **Server Response** (`ack` callback):

```
{
  "ok": true,
  "data": {
    "id": 501,
    "threadId": 102,
    "senderId": 8,
    "message": "Hey everyone, see you at 8:00 AM at the starting point!",
    "isRead": false,
    "createdAt": "2026-08-06T16:46:10.000Z",
    "updatedAt": "2026-08-06T16:46:10.000Z",
    "sender": {
      "id": 8,
      "fullName": "Saqib Usman",
      "profileImage": "https://api.ridewithpals.com/uploads/profiles/user8.jpg"
    }
  }
}
```

5 Fetch Messages List

Fetches paginated message history for a specific thread.

- **Event Name:** `chat:messages:list`
- **Emit Payload:**

```
{
  "threadId": 102,
  "limit": 20,
  "offset": 0
}
```

- **Server Response** (`ack` callback):

```
{
  "ok": true,
  "data": {
    "count": 45,
    "rows": [
      {
        "id": 501,
        "threadId": 102,
        "senderId": 8,
        "message": "Hey everyone, see you at 8:00 AM at the starting point!",
```

```

      "isRead": false,
      "createdAt": "2026-08-06T16:46:10.000Z",
      "sender": {
        "id": 8,
        "fullName": "Saqib Usman",
        "profileImage": "https://api.ridewithpals.com/uploads/profiles/user8.jpg"
      }
    }
  ]
}
}

```

6 Fetch User Threads List

Retrieves all Direct DMs and Activity Group Chats for the current user.

- **Event Name:** `chat:threads:list`
- **Emit Payload:**

```

{
  "search": "",
  "limit": 20,
  "offset": 0
}

```

- **Server Response (`ack` callback):**

```

{
  "ok": true,
  "data": {
    "count": 2,
    "totalUnreadCount": 3,
    "rows": [
      {
        "id": 102,
        "type": "activity",
        "rideId": 45,
        "isGroup": true,
        "title": "Weekend Cycling Tour",
        "otherUser": null,
        "ride": {
          "id": 45,
          "rideName": "Weekend Cycling Tour",
          "clubId": 5,
          "userId": 2
        },
        "lastMessage": {
          "id": 501,

```

```

    "threadId": 102,
    "senderId": 8,
    "message": "Hey everyone, see you at 8:00 AM!",
    "createdAt": "2026-08-06T16:46:10.000Z",
    "sender": {
      "id": 8,
      "fullName": "Saqib Usman",
      "profileImage": "https://api.ridewithpals.com/uploads/profiles/user8.jpg"
    }
  },
  "unreadCount": 2
},
{
  "id": 40,
  "type": "direct",
  "rideId": null,
  "isGroup": false,
  "title": "John Doe",
  "otherUser": {
    "id": 12,
    "fullName": "John Doe",
    "profileImage": "https://api.ridewithpals.com/uploads/profiles/user12.jpg"
  },
  "ride": null,
  "lastMessage": {
    "id": 480,
    "threadId": 40,
    "senderId": 12,
    "message": "Are you ready?",
    "createdAt": "2026-08-06T14:20:00.000Z"
  },
  "unreadCount": 1
}
]
}
}

```

7 Mark Thread Messages as Read

Marks all incoming unread messages in a thread as read.

- **Event Name:** `chat:thread:read`
- **Emit Payload:**

```

{
  "threadId": 102
}

```

- Server Response (`ack` callback):

```
{
  "ok": true
}
```

4. Real-Time Broadcast Listeners (Client Listens Server Emits)

Frontend apps should register listeners on `socket.on(...)` to handle incoming real-time events:

1. Incoming New Message Broadcast

Emitted to all participants in the thread room when a new message is sent.

- Listener Event Name: `chat:message:new`
- Broadcast Payload:

```
{
  "threadId": 102,
  "message": {
    "id": 501,
    "threadId": 102,
    "senderId": 8,
    "message": "Hey everyone, see you at 8:00 AM at the starting point!",
    "isRead": false,
    "createdAt": "2026-08-06T16:46:10.000Z",
    "sender": {
      "id": 8,
      "fullName": "Saqib Usman",
      "profileImage": "https://api.ridewithpals.com/uploads/profiles/user8.jpg"
    }
  }
}
```

```
// Frontend Listener Example
socket.on("chat:message:new", (data) => {
  console.log("New message received in thread:", data.threadId);
  console.log("Message:", data.message);
  // Append message to UI active chat screen
});
```

2. Thread Read Receipt Broadcast

Emitted when a user reads messages in a thread.

- **Listener Event Name:** `chat:thread:read`
- **Broadcast Payload:**

```
{  
  "threadId": 102,  
  "readerId": 12  
}
```

```
// Frontend Listener Example  
socket.on("chat:thread:read", (data) => {  
  console.log(`User ${data.readerId} read thread ${data.threadId}`);  
  // Update read receipts / blue ticks in UI  
});
```